German University in Cairo (GUC)
Mechatronics Department (MCTR)

MCTR911 – Robotics Programming
Winter 2026

Milestone 03 Report

Velocity and Acceleration Kinematics of the UR5e Industrial Robot Arm in a Welding Cell

1. Introduction

This report documents Milestone 03 of the Robotics Programming project. The robot is a Universal Robots UR5e, a 6-DoF serial manipulator with six revolute joints, simulated in MuJoCo and connected to a Python GUI through ROS 2. The chosen industrial application is welding: the arm moves a welding torch along the seam between two workpieces carried by a conveyor.

Milestone 02 produced the DH model, the forward and inverse position kinematics, and the first version of the GUI. Milestone 03 extends the model to the velocity and acceleration levels and checks all equations against the simulator. The techniques used follow Lectures 3 to 5 and Tutorials 3 to 5 of the course (velocity and acceleration kinematics through the Jacobian).

1.1 Milestone requirements and where they are covered

Table 3: *Milestone 03 requirements and the corresponding report sections*

Requirement	Section
Derive forward and inverse velocity kinematics in Python	2 and 3
Derive forward and inverse acceleration kinematics in Python	4
Test position and velocity equations (forward and inverse) on the simulator within a simple tolerance	6
Sense the end-effector position, velocity and acceleration in the simulator and compare with the equations	6
Continue building the GUI with the environment components (conveyor, tools, people, etc.)	5
Deliverables: validation video, report, code, GUI videos, equation-test videos	Repository

2. Forward Velocity Kinematics

The Jacobian columns are populated using the cumulative transformations ${}^{0}T_{i}$ of the DH model from Milestone 02. The forward velocity kinematics relate the joint velocities to the end-effector velocities (Lecture 5 and Tutorial 5):

$$\dot{X} = J(q) \cdot \dot{q}$$

where $\dot{X} = [v_x, v_y, v_z, \omega_x, \omega_y, \omega_z]^T$ is the end-effector twist, $\dot{q}$ is the vector of joint velocities and J is the 6×6 Jacobian. Each row of J shows how all joints contribute to one velocity component, and each column shows how one joint contributes to all components.

2.1 Jacobian by the general (geometric) approach

Lecture 5 gives a general rule that works for any serial arm using only the cumulative DH matrices. Each joint is actuated alone while the rest of the robot is treated as rigid. For a revolute joint i, the end effector rotates about the axis ${}^0Z_{i-1}$, which gives:

$$J_{vi} = {}^0Z_{i-1} \times ({}^0O_n - {}^0O_{i-1}) \qquad J_\omega_i = {}^0Z_{i-1}$$

For a prismatic joint the rule would be $J_{vi} = {}^0Z_{i-1}$ and $J_\omega_i = 0$. The UR5e has only revolute joints, so only the first rule is used. ${}^0Z_{i-1}$ is the third column of ${}^0T_{i-1}$, ${}^0O_{i-1}$ is its translation column and ${}^0O_n$ is the end-effector position taken from ${}^0T_6$. The six columns $[J_{vi} ; J_\omega_i]$ form the 6×6 Jacobian.

Implementation: `compute_jacobian(q)` loops over the six joints, reads ${}^0Z_{i-1}$ and ${}^0O_{i-1}$ from `forward_kinematics(q)`, and fills column i with `np.cross(Z_prev, O_n - O_prev)` in the top three rows and `Z_prev` in the bottom three rows. `forward_velocity_kinematics(q, q_dot)` then returns $J \cdot \dot{q}$.

3. Inverse Velocity Kinematics

The inverse velocity kinematics compute the joint velocities that produce a desired end-effector velocity, by making $\dot{q}$ the subject of the equation: $\dot{q} = J^{-1} \dot{X}$. Tutorial 5 distinguishes three cases according to the number of joint DoF compared with the task-space DoF:

Table 4: *Choice of inverse according to the Jacobian shape (Tutorial 5)*

Case	Jacobian	Inverse used
Joints < task DoF (under-actuated)	tall	Pseudo-inverse $J^+ = (J^TJ)^{-1}J^T$
Joints = task DoF (exact system)	square	Inverse J^{-1}
Joints > task DoF (over-actuated)	wide	Pseudo-inverse $J^+ = J^T(JJ^T)^{-1}$

Two of these cases occur for the UR5e in our implementation:

- **Full twist (6 task DoF, 6 joints):** J is square, so $\dot{q} = J^{-1}\dot{X}$.
- **Position only (3 task DoF, 6 joints):** J is the top 3×6 block, which is the over-actuated case, so $\dot{q} = J^+\dot{x}$ with $J^+ = J^T(JJ^T)^{-1}$. This is the mode used by the GUI, which commands Cartesian linear velocities only.

3.1 Singularities and the damped fallback

The inverse does not exist when $\det J = 0$. Tutorial 5 calls this a singularity: the arm loses the ability to move the end effector in some direction and the joint velocities needed grow without bound. The upright pose [0, −π/2, 0, −π/2, 0, 0] is such a case: our tests give $\det J \approx 10^{-83}$ there, so that pose is not used for inverse tests.

As an extension beyond the tutorial, `inverse_velocity_kinematics` guards against singular poses. If $|\det J| < 10^{-4}$ it replaces J^{-1} by the damped pseudo-inverse

$$J^* = J^T (J J^T + \lambda^2 I)^{-1}, \quad \lambda = 10^{-3}$$

which stays bounded near a singularity at the cost of a small tracking error. The position-only mode always uses the damped form with the 3×3 identity. Away from singularities, λ is small enough that J* is practically equal to the exact pseudo-inverse of the tutorial.

4. Forward and Inverse Acceleration Kinematics

4.1 Forward acceleration kinematics

Starting from $\dot{X} = J\dot{q}$ and applying the product rule (Tutorial 5, slide 18), the end-effector acceleration is:

$$\ddot{X} = J \cdot \dot{q} + J \cdot \ddot{q}$$

Because J depends on the joint angles, which depend on time, its derivative follows the chain rule:

$$\dot{J} = \Sigma_i \, (\partial J / \partial q_i) \cdot \dot{q}_i$$

For the UR5e, writing these 36 entries by hand is impractical, so the code computes $\dot{J}$ numerically. The sum above is the directional derivative of J along $\dot{q}$, which a central difference approximates to second order:

$$\dot{J} \approx [\, J(q + \dot{q} \cdot h) - J(q - \dot{q} \cdot h) \,] \,/\, (2h), \quad h = 10^{-4}$$

This is implemented in `compute_jacobian_derivative(q, q_dot, dt=1e-4)`, and `forward_acceleration_kinematics` returns $\dot{J}\dot{q} + J\ddot{q}$.

4.2 Inverse acceleration kinematics

Making $\ddot{q}$ the subject of the acceleration equation gives:

$$\ddot{q} = J^{-1} \, [\, \ddot{X} - \dot{J} \cdot \dot{q} \,]$$

`inverse_acceleration_kinematics` follows the same rules as the velocity case: J^{-1} for the full 6×6 problem, with the damped pseudo-inverse if $|\det J| < 10^{-4}$, and the damped over-actuated pseudo-inverse of the top 3×6 block for the position-only problem (using only the first three entries of $\dot{J}\dot{q}$).

5. Simulation Environment and GUI

5.1 System architecture

The system has two processes connected by ROS 2 topics. The simulation node runs the MuJoCo model and the passive viewer. The GUI node (Tkinter) shows telemetry, sends commands and calls the kinematics functions. The topics are:

Table 5: *ROS 2 topics used between the GUI and the simulation*

Topic	Type	Direction	Purpose
/joint_commands	Float64MultiArray	GUI → sim	Six target joint angles
/joint_states	JointState	sim → GUI	Measured joint positions and velocities
/welding_active	Bool	GUI → sim	Torch on / off
/weld_clear	Empty	GUI → sim	Erase the weld bead
/conveyor_active	Bool	GUI → sim	Conveyor on / off
/auto_weld	Empty	GUI → sim	Start or abort the automatic weld test

Topic	Type	Direction	Purpose
/auto_weld_status	String	sim → GUI	IDLE, WAITING, MOVING, WELDING, DONE or ERROR

5.2 The welding cell

The cell is defined in `welding_cell.xml`, which includes the Menagerie `ur5e.xml`. The model contains a table, a conveyor belt with side rails and yellow station markers, and two box-shaped workpieces side by side whose shared face is the weld seam. The welding torch (neck, nozzle and a `torch_tip` site) is attached to the `wrist_3_link` at load time using MuJoCo's `MjSpec`, and gravity compensation is enabled on all bodies so that the arm tracks its commands without sagging.

The conveyor moves both boxes together at 0.04 m/s and wraps them back after 0.35 m. The weld bead is recorded in the local frame of each box, so it travels with the box on the conveyor. A bead point is added when the torch is on and the tip is within 15 mm of a box surface, with a minimum spacing of 4 mm and a radius of 4 mm; a blue sphere marks the arc.

The workcell also includes an inspection chair and an operator safety boundary placed around the robot's working area.

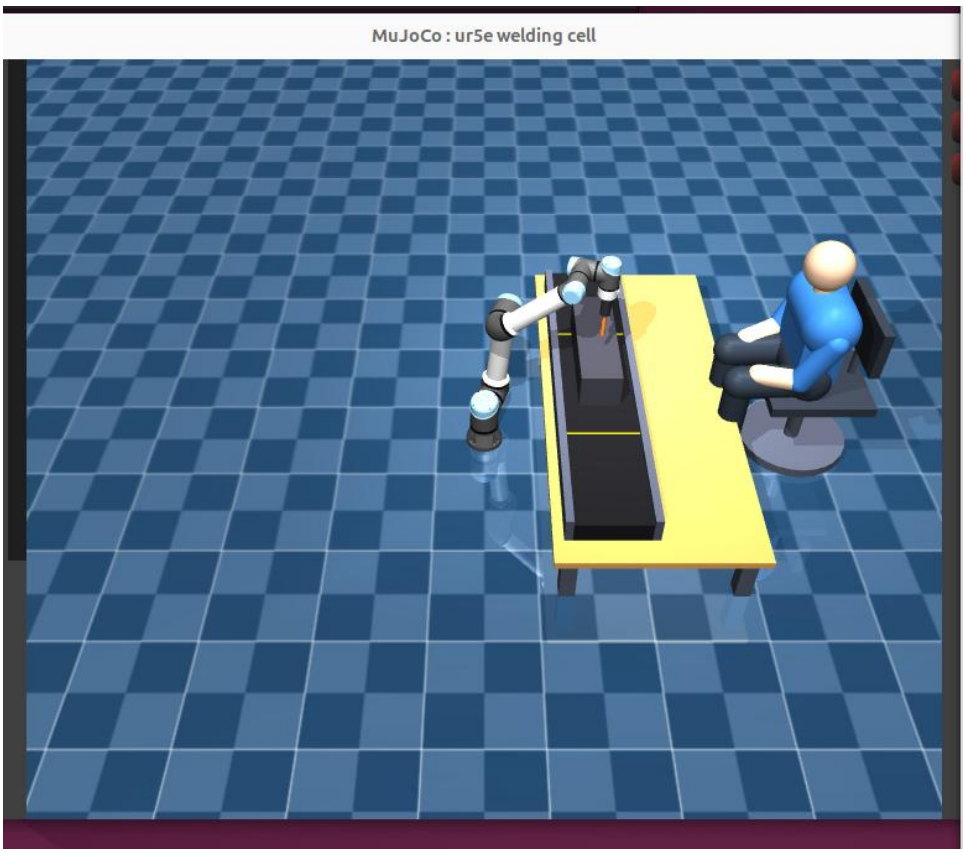


Figure 1: *Overview of the MuJoCo welding cell: UR5e arm, table, and the conveyor carrying the workpiece.*

5.3 GUI

The GUI window has five panels:

1. **Kinematic telemetry:** end-effector position, linear velocity, angular velocity, linear acceleration and the six joint velocities. Position, velocity and acceleration are computed with `get_end_effector_position`, `forward_velocity_kinematics` and `forward_acceleration_kinematics` from the joint state received on `/joint_states`; $\ddot{q}$ is estimated by differencing the measured joint velocities.

2. **Cartesian position control:** X, Y, Z sliders solved by `inverse_kinematics_position` (the Milestone 02 solver) and sent as joint commands.
3. **Joint position control:** six joint sliders (forward kinematics path) with a reset-to-upright button.
4. **Cartesian velocity input:** sliders for v_x, v_y, v_z. The 'Step Velocity' button applies $\dot{q} = J^{+}\dot{x}$ from `inverse_velocity_kinematics` (position-only mode) for 0.1 s.
5. **Welding and workcell controls:** torch on/off, clear bead, run/abort automatic weld with live status, and conveyor on/off.

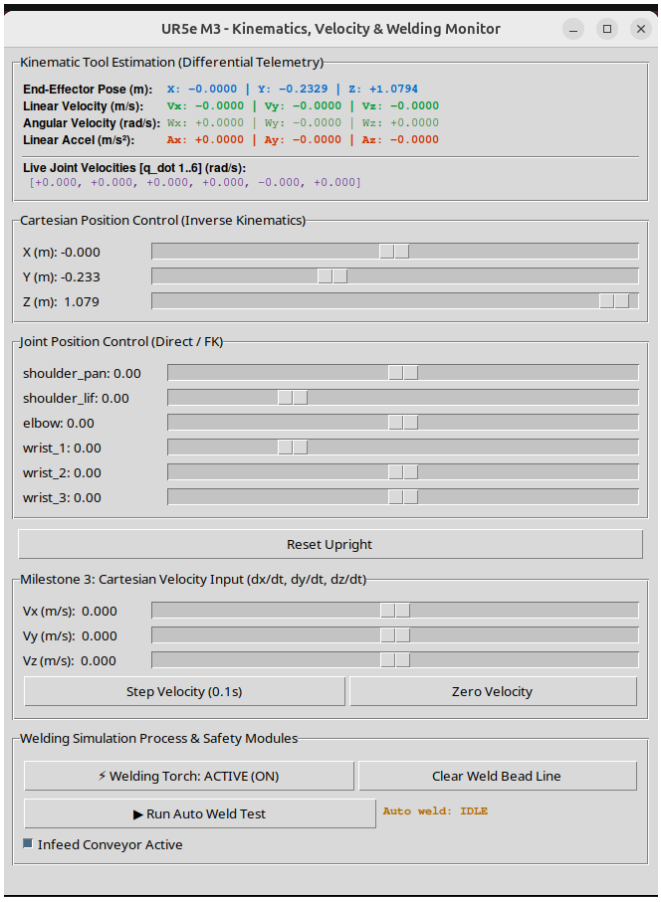


Figure 2: *Complete GUI window with all five panels.*

6. Validation Against the Simulator

The milestone asks that the equations be tested in the simulator by inputting joint angles, angular velocities and angular accelerations, sensing the end-effector position, velocity and acceleration, and checking that the inverse process returns the same values within a simple tolerance. The script `validate_kinematics_mujoco_v2.py` does this, and every 'measured' value is read from MuJoCo and not from our own equations.

6.1 Method

Sensing in MuJoCo

- The state q, $\dot{q}$, $\ddot{q}$ is written into MuJoCo and `mj_inverse` is called, which computes the body accelerations.
- The end-effector position is read from the `attachment_site`. The twist is read with `mj_objectVelocity` and the acceleration with `mj_objectAcceleration`, both in the world

frame. MuJoCo returns [angular, linear], and the script reorders them to [linear, angular] to match our Jacobian.

- As an independent check of the linear acceleration, the site position is evaluated along $q(t) = q + \dot{q}t + \frac{1}{2}\ddot{q}t^2$ and differentiated twice numerically with step 10^{-4} s.

Forward and inverse tests

- **Forward:** FK(q), $J\dot{q}$ and $\dot{J}\dot{q} + J\ddot{q}$ are compared with the sensed position, twist and acceleration.
- **Inverse:** the sensed twist and accelerations from MuJoCo are fed into the inverse equations, and the recovered $\dot{q}$ and $\ddot{q}$ are compared with the values that were applied. The angular acceleration used in the inverse test is the one sensed by MuJoCo, so the test does not reuse our own forward result.

Test configurations

Three configurations were used, none at a singularity. The upright pose was avoided because J is singular there and J^{-1} does not exist (Section 3.1). The condition number of J is printed for each case.

Table 6: *Test cases (joint values in rad, rad/s and rad/s²)*

Case	q	$\dot{q}$	$\ddot{q}$
Config A	[0.3, −1.0, 1.2, −0.5, 0.4, 0.1]	[0.2, −0.1, 0.3, −0.2, 0.1, 0.0]	[0.5, −0.4, 0.8, −0.6, 0.3, 0.2]
Config B	[−0.5, −0.7, 0.9, −1.2, 1.0, −0.5]	[−1.0, 1.2, −0.8, 1.5, −1.1, 0.9]	[2.0, −1.5, 1.0, −2.0, 1.5, 1.0]
Config C	[1.0, −1.8, 1.5, −1.0, −0.8, 0.6]	[0.4, 0.3, −0.5, 0.2, 0.6, −0.3]	[−0.3, 0.6, 0.2, −0.5, 0.1, 0.4]

Tolerances

Table 7: *Tolerances used in the script*

Quantity	Tolerance	Reason
End-effector position	2×10^{-3} m	Mesh versus DH link-length rounding gives about 1.2 mm
Linear and angular velocity	5×10^{-3} m/s, rad/s	Equations are exact; allows numerical noise
Linear acceleration	2×10^{-2} m/s²	Includes the numerical $\dot{J}$ and the finite-difference reference
Recovered joint velocity	5×10^{-2} rad/s	Position error enters through J
Recovered joint acceleration	5×10^{-1} rad/s²	Amplifies position and $\dot{J}$ errors

6.2 Results

The tables below summarise the output of the final run of the validation script, in which all checks passed (RESULT: ALL CHECKS PASSED). Each value is the error norm between our equations and the value sensed in MuJoCo.

Table 8: *Forward test: equations versus MuJoCo (error norms)*

Check	Config A	Config B	Config C	Tolerance
cond(J)	13.8	14.5	12.3	–
Position FK(q) (m)	1.110×10^{-3}	1.154×10^{-3}	9.825×10^{-4}	2×10^{-3}
Linear velocity $J\dot{q}$ (m/s)	1.779×10^{-4}	1.408×10^{-3}	3.999×10^{-4}	5×10^{-3}
Angular velocity $J\dot{q}$ (rad/s)	1.775×10^{-16}	1.662×10^{-15}	5.495×10^{-16}	5×10^{-3}

Check	Config A	Config B	Config C	Tolerance
Linear acceleration (m/s²)	3.907×10^{-4}	4.354×10^{-3}	2.915×10^{-4}	2×10^{-2}

Table 9: *Inverse test: MuJoCo readings through the inverse equations (error norms)*

Check	Config A	Config B	Config C	Tolerance
$\dot{q} = J^{-1}\,\dot{x}$(sensed) (rad/s)	7.610×10^{-4}	5.759×10^{-3}	1.032×10^{-3}	5×10^{-2}
$\ddot{q} = J^{-1}(\ddot{x} - \dot{J}\dot{q})$ (rad/s²)	1.575×10^{-3}	2.757×10^{-2}	1.649×10^{-3}	5×10^{-1}

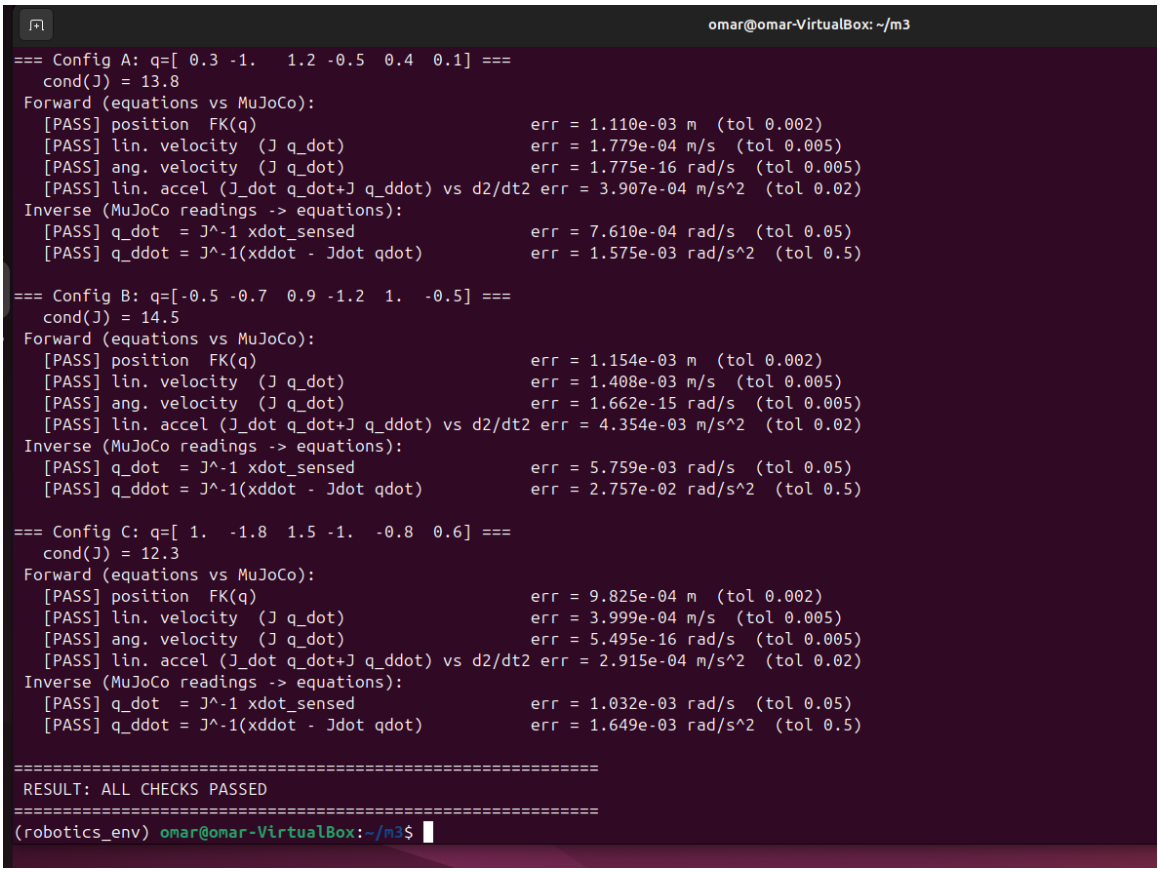


Figure 3: *Console output of the validation script for Configs A, B and C, ending with the final RESULT line (all checks PASS).*

6.3 Discussion of the results

All checks passed in all three configurations. The largest forward error is the position error of about 1 mm (0.98 to 1.15 mm), which is within the 2 mm tolerance and consistent with the roughly 1.2 mm offset between the DH link lengths and the mesh geometry of the MuJoCo model. The linear velocity errors are between 1.8×10^{-4} and 1.4×10^{-3} m/s, and the linear acceleration errors between 2.9×10^{-4} and 4.4×10^{-3} m/s², which are 28% and 22% of their tolerances at most. The linear acceleration is the least exact forward quantity because it uses the numerical $\dot{J}$ and is compared with a finite-difference reference. The angular velocity errors are of the order of 10^{-15} rad/s, which is rounding error only: the angular rows of J are the joint axes themselves, so they match MuJoCo exactly.

Config B has the largest error in almost every row. It also has the highest condition number (14.5) and the largest joint velocities and accelerations of the three test cases, so it excites the numerical $\dot{J}$ and the position offset most. The differences in cond(J) between the configurations are small (12.3 to 14.5) and all three are far from a singularity, so the inverse errors follow the size of the inputs more than the conditioning. In the inverse test the recovered $\ddot{q}$ error is about 1.6 to 5 times larger

than the $\dot{q}$ error (for example 2.8×10^{-2} rad/s² against 5.8×10^{-3} rad/s in Config B), because $\ddot{q}$ also inherits the error of $\dot{J}\dot{q}$. Even so, it stays well below its tolerance of 0.5 rad/s². No check failed, so no change to the equations or tolerances was needed.